Folha de consulta — Aula 09

Aula 09 — Inimigos, horda, vida e o objetivo do jogo

Jogos Digitais II · 21/10/2026 · Lab 3.54

Prof. Ezéfferth C. A. Fernandes

Hoje o professor constrói na frente da turma, na vila da Aula 08, as quatro peças que todo jogo da turma precisa: um inimigo que persegue, a horda que traz mais inimigos, a vida do jogador e o fim da partida, com vitória e derrota. Depois é a sua vez, no seu jogo.

Esta folha tem o que foi mostrado, na mesma ordem. Se você se perdeu em algum ponto da demonstração, retome por aqui.

A entrega de hoje vai até o tipo **Sobrevivência**, que é o fim da Parte 4, qualquer que seja o seu tipo de jogo. As fichas dos outros sete tipos estão no fim da folha, como próxima etapa: Abate e Chefe pedem o ataque, que ainda vai ser mostrado em aula.

Você parte do jogo da Aula 08, com a tag `aula-08`. Quem escolheu o modo plataforma começa pela seção "Modo plataforma", no fim.

Checkpoints

Ao terminar a	O jogo faz	Se não fizer
Parte 1	um inimigo vermelho vem atrás de você	o `Jogador` está no grupo `player`?
Parte 2	inimigos nascem nas pontas da rua, cada vez mais rápido	o sinal `timeout` do `Horda` foi conectado?
Parte 3	encostou, perde 1 de vida e fica meio transparente por 1 segundo	a `Hurtbox` enxerga a camada 3?
Parte 4	sem vida, aparece "Você perdeu!"; aguentando 60 s, "Você venceu!"	as duas cenas foram salvas em `scenes/`?

Parte 1 — O inimigo

1.1 As camadas de colisão

Até a Aula 08 tudo estava na camada 1. Agora são três, e cada objeto diz em qual está (Layer) e o que enxerga (Mask).

Camada	Quem está nela
1	o mundo: muro, casas, árvores (o TileMap)
2	o jogador
3	os inimigos

Dê nome às camadas, para o Inspetor mostrar o nome em vez do número: **Project** → **Project Settings** → **Layer Names** → **2D Physics**, camada 1 `mundo`, 2 `jogador`, 3 `inimigo`.

Logo acima de **2D Physics** fica **2D Render**, com os mesmos campos. Nome escrito lá não dá erro, mas também não aparece em Collision: o Inspetor continua mostrando 1, 2, 3. Se isso acontecer, apague os nomes de 2D Render e escreva em 2D Physics.

Abra `entities/jogador.tscn` com dois cliques no FileSystem, selecione o `Jogador` e, em **Collision**, deixe a **Layer** só na 2 e a **Mask** só na 1.

Ainda no `Jogador`, abra a aba **Node** (ao lado do Inspetor) → **Groups**, digite `player` e clique em **Add**. Se aparecer a opção **Global**, tanto faz marcar ou não: o inimigo acha o jogador do mesmo jeito. Confira na árvore: o `Jogador` ganha um ícone de grupo ao lado do nome. Salve com Ctrl+S.

> **O grupo vai na cena do jogador, não na vila.** Com a `vila.tscn` aberta, é fácil clicar no nó errado e pôr o grupo `player` no inimigo de teste. Aí o inimigo persegue a si mesmo e fica parado. Antes de mexer em qualquer coisa, olhe o nome da aba ativa, no alto do editor: abrir outra cena não fecha a anterior, e a edição vai para a aba que estiver na frente.

1.2 A cena

Scene → **New Scene** → **Other Node** → `CharacterBody2D`. Renomeie para `Inimigo` e salve como `entities/inimigo.tscn`. **Motion Mode** → **Floating**, como o jogador.

Filho	Como
`Sprite2D`	**Texture** `sprites/personagem.png`; **Animation** → **Hframes 4, Vframes 4**; **Visibility** → **Modulate** vermelho; **Ordering** → **Z Index 1**
`CollisionShape2D`	`RectangleShape2D` de 10 × 6 na posição (0, 5), nos pés, como o do jogador

O filho é `Sprite2D`, não `AnimatedSprite2D` como no jogador. O inimigo não precisa animar, e um `AnimatedSprite2D` sem quadros não desenha nada: o inimigo existe, persegue, e você não o vê.

Para pôr a imagem no **Texture**, há dois caminhos. Arrastar: selecione o `Sprite2D`, ache no FileSystem o `sprites/personagem.png` e arraste-o até o campo `<empty>` do **Texture**, no Inspetor. Ou pelo menu: a setinha ao lado de `<empty>` → **Quick Load** → digite `personagem`. Deu certo quando o campo mostra a miniatura da imagem. Se aparecerem os 16 bonecos juntos, faltou o **Hframes** e o **Vframes**.

Para o tamanho da colisão: no `CollisionShape2D`, campo **Shape** → **New RectangleShape2D**. Depois clique no próprio retângulo, dentro do campo **Shape**, para abrir as propriedades dele: é ali que fica o **Size** (10 × 6). A posição (0, 5) é do `CollisionShape2D`, em **Transform** → **Position**. Sem ajustar, o retângulo fica com 20 × 20, maior que o jogador, e o inimigo enrosca onde você passa.

O inimigo é o seu próprio personagem pintado de vermelho. A arte do seu tema entra na Aula 11; até lá, retângulo ou boneco vermelho servem.

No `Inimigo`, o nó de cima da árvore: **Layer** só na 3, **Mask** na 1 e na 3. Grupo `inimigo`, no `Inimigo` e não no `Sprite2D`: quem encosta nas outras coisas é o corpo.

> **Por que o inimigo enxerga a camada 3.** Sem ela, todos andam para o mesmo ponto e viram um inimigo só, empilhados no mesmo pixel. Enxergando os outros, eles se empurram e cercam você.

> **Por que o inimigo não enxerga a camada 2.** Ele passa por dentro de você, de propósito. Se colidisse, pararia na borda do seu corpo e nunca entraria na `Hurtbox` da Parte 3, que é menor que o corpo: você não perderia vida. E cinco inimigos encostados viram uma parede da qual você não sai. O toque aparece na tela de outro jeito: você fica meio transparente por um segundo.

1.3 O script

`scripts/inimigo.gd`, ligado ao `Inimigo`:

```
extends CharacterBody2D

@export var velocidade: float = 40.0


func _physics_process(_delta: float) -> void:
    var jogador := get_tree().get_first_node_in_group("player") as Node2D
    if jogador == null:
        return

    velocity = global_position.direction_to(jogador.global_position) * velocidade
    move_and_slide()
```

`direction_to` devolve a seta de tamanho 1 que aponta daqui até o jogador. Vezes a velocidade, é o passo do quadro. O inimigo anda a 40 e você a 70: dá para fugir.

Teste: arraste um `inimigo.tscn` para a vila, num canto, e rode. Ele vem atrás de você e para nas casas. Se ele aparece mas não sai do lugar, o grupo `player` não está no `Jogador`. Para ver a colisão de todo mundo enquanto testa, ligue **Debug** → **Visible Collision Shapes**.

Depois de testar, apague-o: os próximos nascem sozinhos.

Parte 2 — A horda

2.1 De onde eles vêm

Na `vila.tscn`, um `Node2D` chamado `Nascedouros` e, dentro dele, quatro `Marker2D` nas pontas da rua, por dentro do muro. O nome tem que ser exatamente `Nascedouros`, com N maiúsculo: o script procura o nó por ele. Para criar: botão direito na `Vila` → **Add Child Node** → `Node2D`; depois, botão direito no `Nascedouros` → **Add Child Node** → `Marker2D`, quatro vezes.

Marker2D	Posição
`Esquerda`	(40, 200)
`Direita`	(600, 200)
`Cima`	(312, 40)
`Baixo`	(312, 344)

Se o seu mapa é outro, ponha os pontos onde fizer sentido, sempre em chão livre. Ponto dentro de casa ou de árvore faz o inimigo nascer preso.

2.2 O relógio da horda

Filho da `Vila`: um `Timer` chamado `Horda`, **Wait Time 2**, **Autostart** ligado.

2.3 O script da vila

`scripts/vila.gd`, ligado à `Vila`: botão direito no nó de cima da árvore da vila → **Attach Script**, caminho `res://scripts/vila.gd`. Se o seu nó de cima tem outro nome, tudo bem; o script vai nele mesmo assim. Por enquanto só a horda; as Partes 3 e 4 completam este arquivo.

```
extends Node2D

@export var cena_inimigo: PackedScene
```

```
@onready var horda: Timer = $Horda


func _on_horda_timeout() -> void:
    var inimigo := cena_inimigo.instantiate()
    inimigo.global_position = $Nascedouros.get_children().pick_random().global_position
    add_child(inimigo)

    # A horda aperta: cada inimigo chega um pouco mais cedo que o anterior.
    horda.wait_time = max(0.5, horda.wait_time * 0.95)
```

Antes de rodar, duas ligações:

1. **O sinal.** Selecione o `Horda` e abra **Node** → **Signals**. Dê dois cliques em `timeout` e conecte à `Vila`. O método sugerido, `_on_horda_timeout`, é o que o script usa.
2. **A cena do inimigo.** Selecione a `Vila` e arraste `entities/inimigo.tscn` para o campo **Cena Inimigo** no Inspetor. Vazio, o jogo quebra com erro em `instantiate()`.

O `$` acha o nó pelo nome exato, com maiúscula e minúscula. Se o nó se chama `nasce` ou `nascedouros`, o `$Nascedouros` devolve nada e o jogo para com "Cannot call method 'get_children' on a null value". A correção é renomear o nó, não mudar o script: as próximas partes usam os mesmos nomes.

`pick_random()` sorteia um dos quatro pontos. O `0.95` é o primeiro número de design do seu jogo: com `0.98`, a pressão demora a chegar; com `0.9`, chega na metade do tempo.

Parte 3 — Vida e dano

3.1 A área que sente o toque

Filho do `Jogador`: uma **`Area2D`** chamada `Hurtbox`, com um `CollisionShape2D` filho (`CircleShape2D`, raio 6, posição (0, 2)). Na `Hurtbox`: **Layer** nenhuma, **Mask** só na 3.

O corpo do jogador não enxerga os inimigos (a Mask dele é só a 1), então eles passam por dentro de você. Quem percebe é a `Hurtbox`.

A `Area2D` nasce com Layer e Mask na camada 1: desmarque as duas antes de marcar a 3. E o círculo fica dentro da `Hurtbox`, não solto embaixo do `Jogador`:

```
Jogador
+-- AnimatedSprite2D
+-- CollisionShape2D
+-- Camera2D
+-- Hurtbox
    +-- CollisionShape2D
```

3.2 O script do jogador

Em `scripts/jogador.gd`, acrescente a vida no começo, a função `_process` e a função `levar_dano` no fim. O `_physics_process` da Aula 08 não muda. O arquivo completo fica assim:

```
extends CharacterBody2D

@export var velocidade: float = 70.0
@export var vida: int = 3

var invencivel := false
```

```
@onready var anim: AnimatedSprite2D = $AnimatedSprite2D
@onready var hurtbox: Area2D = $Hurtbox


func _process(_delta: float) -> void:
    # Tem inimigo encostando em mim?
    if not invencivel and hurtbox.has_overlapping_bodies():
        levar_dano()


func _physics_process(_delta: float) -> void:
    var direcao := Input.get_vector("left", "right", "up", "down")
    velocity = direcao * velocidade
    move_and_slide()

    if direcao == Vector2.ZERO:
        anim.stop()
        return

    if abs(direcao.x) > abs(direcao.y):
        if direcao.x > 0:
            anim.play("direita")
        else:
            anim.play("esquerda")
    else:
        if direcao.y > 0:
            anim.play("baixo")
        else:
            anim.play("cima")


func levar_dano() -> void:
    vida -= 1
    if vida <= 0:
        get_tree().current_scene.perder()
        return

    # Um segundo de invencibilidade, meio transparente.
    invencivel = true
    modulate.a = 0.4
    await get_tree().create_timer(1.0).timeout
    modulate.a = 1.0
    invencivel = false
```

Três ideias:

- **A cada quadro, o jogador pergunta: tem inimigo encostando em mim?** Quem responde sim ou não é o `has_overlapping_bodies()`. É uma linha, e vale para um inimigo ou para cinquenta.
- **Sem invencibilidade, você morre num piscar.** O toque dura vários quadros, e cada quadro tiraria 1 de vida. É o mesmo problema do `hurt` da Aula 06.
- **`await ... timeout`** espera um segundo sem travar o jogo e depois continua da linha seguinte.

A função `perder()` ainda não existe: ela vem na Parte 4. Se rodar agora, o jogo para com erro quando a vida acaba.

Parte 4 — O fim da partida

4.1 As duas telas de fim

Nova cena, **User Interface** (nó `Control`), renomeie para `Fim`. Filho: um `Label` com **Layout** → **Anchors Preset** → **Full Rect**, **Horizontal** e **Vertical Alignment** → **Center**, e **Theme Overrides** → **Font Sizes** → **48**. Texto em duas linhas:

```
Você perdeu!
Enter para jogar de novo
```

Script `scripts/fim.gd`, ligado ao `Fim`:

```
extends Control


func _process(_delta: float) -> void:
    if Input.is_action_just_pressed("ui_accept"):
        get_tree().change_scene_to_file("res://scenes/vila.tscn")
```

A linha do `get_tree()` tem dois recuos de Tab: um por estar dentro da `func`, outro por estar dentro do `if`. Com um só, a Godot acusa erro de indentação na linha do `if`.

Salve como `scenes/derrota.tscn`. A janela de salvar abre na última pasta usada, que costuma ser `entities/`: confira que o caminho no alto dela diz `res://scenes/`. Cena salva em outra pasta não dá erro ao salvar, só quando a partida acaba: a troca de cena falha, o Debugger acusa o caminho e o jogo fica parado na vila.

Depois, em **Scene** → **Save Scene As**, salve uma cópia como `scenes/vitoria.tscn`. A aba ativa passa a ser a `vitoria.tscn`: troque o texto do `Label` para "Você venceu!" e salve de novo com Ctrl+S. As duas cenas usam o mesmo script.

4.2 O relógio da partida e o placar

Filho da `Vila`: outro `Timer`, chamado `Relogio`, **Wait Time 60**, **One Shot** e **Autostart** ligados. Conecte o sinal como fez com o `Horda`: selecione o `Relogio`, **Node** → **Signals**, dois cliques em `timeout`, conecte à `Vila`, método `_on_relogio_timeout`. Sem a conexão não há erro: o tempo zera e a partida segue para sempre.

Filho da `Vila`: um `CanvasLayer` chamado `HUD`, com dois `Label` dentro, `Vida` e `Tempo` (**Font Sizes 32**; o `Tempo` um pouco abaixo do `Vida`). O `CanvasLayer` desenha por cima de tudo e não anda com a câmera.

Antes do script, confira os nomes na árvore da vila. O script usa `$Jogador`, `$Horda`, `$Relogio`, `$HUD/Vida` e `$HUD/Tempo`, com essa grafia. Se o seu jogador aparece na vila como `jogador` ou `player`, renomeie o nó na vila. Quando o `$Jogador` não acha ninguém, o jogo para logo ao abrir, com erro de acesso a `vida` numa `null instance`.

```
Vila
+-- chao, objetos (as camadas da Aula 08)
+-- Jogador
+-- Nascedouros
+-- Horda
+-- Relogio
+-- HUD
    +-- Vida
    +-- Tempo
```

4.3 O script da vila, completo

```
extends Node2D

@export var cena_inimigo: PackedScene
```

```
@onready var horda: Timer = $Horda
@onready var relogio: Timer = $Relogio
@onready var jogador := $Jogador
@onready var texto_vida: Label = $HUD/Vida
@onready var texto_tempo: Label = $HUD/Tempo


func _process(_delta: float) -> void:
    texto_vida.text = "Vida: %d" % jogador.vida
    texto_tempo.text = "Tempo: %d" % ceil(relogio.time_left)


func _on_horda_timeout() -> void:
    var inimigo := cena_inimigo.instantiate()
    inimigo.global_position = $Nascedouros.get_children().pick_random().global_position
    add_child(inimigo)

    # A horda aperta: cada inimigo chega um pouco mais cedo que o anterior.
    horda.wait_time = max(0.5, horda.wait_time * 0.95)


func _on_relogio_timeout() -> void:
    vencer()


# call_deferred: troca de cena só no fim do quadro, quando a física terminou.
func vencer() -> void:
    get_tree().change_scene_to_file.call_deferred("res://scenes/vitoria.tscn")


func perder() -> void:
    get_tree().change_scene_to_file.call_deferred("res://scenes/derrota.tscn")
```

`vencer()` e `perder()` são as duas portas de saída da partida. Qualquer peça do jogo que precisar encerrar chama uma delas com `get_tree().current_scene.vencer()`: a cena atual é a vila. É isso que as fichas do fim da folha usam.

Teste: fique parado e perca as três vidas; depois fuja por 60 segundos. As duas telas aparecem, e o Enter recomeça.

Sobrevivência — a entrega de hoje

O que está rodando já é o tipo Sobrevivência, com 60 segundos em vez de 3 minutos. Quem tirou Sobrevivência põe **Wait Time 180** no `Relogio` e o objetivo do jogo está pronto. O que sobra é design: o `0.95` da horda e a velocidade do inimigo decidem se três minutos são fáceis ou impossíveis.

Quem tirou outro tipo entrega o mesmo jogo, com o `Relogio` em 60 s. A ficha do seu tipo, no fim da folha, troca o que o relógio faz e acrescenta uma peça; ela entra na próxima etapa, em cima do que você entrega hoje.

A entrega é a tag `aula-09` com as Partes 1 a 4 funcionando: inimigo, horda, vida, as duas telas de fim e o placar. Os passos estão em "Commit e tag", logo adiante.

Modo plataforma

O seu sobrevivente da Aula 06 já tem vida, `hurt` e `morrer()`, e o zumbi já ataca. O que falta é a horda e o fim da partida.

1. **A horda** é a Parte 2, com os `Nascedouros` nas duas pontas da fase, um pouco acima do chão, e

`cena_inimigo` apontando para o seu `zumbi_*.tscn`. Antes do `add_child`, acrescente `inimigo.distancia_de_visao = 5000.0`: sem isso, o zumbi nascido na ponta fica dormindo até você chegar perto.

2. **O fim** é a Parte 4 inteira. No `morrer()` do sobrevivente, chame `get_tree().current_scene.perder()`. A entrega de hoje termina aqui, como no modo horda.
3. **A ficha do seu tipo**, na próxima etapa, vale igual. O ataque do Abate e do Chefe você já tem: no lugar do `levar_golpe()`, chame o `inimigo_morreu(self)` da vila dentro do `morrer()` do zumbi.

O exemplo pronto

O resultado das Partes 1 a 4 está no repositório da disciplina, na branch `aula-09`: a vila com o inimigo, a horda, a vida, o placar e as duas telas de fim, com os mesmos nomes desta folha. As fichas dos tipos de jogo não estão lá; elas ficam no seu projeto.

Sem git: abra `github.com/ezefferth/construcao-jogo-godot/tree/aula-09`, use **Code** → **Download ZIP**, descompacte numa pasta à parte e, na Godot, **Import** apontando para o `project.godot`.

Com git, também numa pasta à parte:

```
git clone -b aula-09 https://github.com/ezefferth/construcao-jogo-godot.git
↪  exemplo-aula-09
```

Três cuidados:

- A página inicial do repositório abre na `main`, que parou na Aula 05. Troque para `aula-09` no seletor de branch antes de baixar o ZIP.
- O exemplo é para consulta. Não copie a pasta por cima do seu projeto: você perde o seu tema e o seu tipo de jogo. Abra os dois e compare cena por cena, script por script.
- Ao copiar um trecho de script, confira os nomes. Se o seu jogador se chama `player` e não `Jogador`, o `$Jogador` copiado devolve nulo e o jogo para na primeira linha que usa a vida.

Commit e tag

DEVLOG com a entrada de hoje — o que fez, o que travou, como resolveu — e:

```
git add .
git commit -m "aula-09: inimigos, horda, vida e o objetivo do jogo"
git tag aula-09
git push origin main --tags
```

Se sobrar tempo

1. **Game feel no dano.** Em vez de ficar transparente, o jogador pisca: troque o `modulate.a = 0.4` por um laço que alterna `visible` cinco vezes, com `await get_tree().create_timer(0.1).timeout` entre uma troca e outra.
2. **Inimigos diferentes.** Duplique o `inimigo.tscn` como `inimigo_rapido.tscn`, com velocidade 65 e Modulate laranja, e sorteie entre as duas cenas na horda com `[cena_inimigo, cena_rapido].pick_random()`.

3. **A tela de início.** Uma terceira cena com o nome do seu jogo e "Enter para começar", com o mesmo `fim.gd`, marcada como cena principal.

As armadilhas desta aula

- **Inimigo parado.** O `Jogador` não está no grupo `player`; o inimigo não acha alvo e não reclama. Ou o grupo `player` foi parar no próprio inimigo, e ele persegue a si mesmo.
- **Inimigo invisível, mas o jogo roda.** O filho dele é um `AnimatedSprite2D` vazio, ou o `Sprite2D` ficou sem **Texture**.
- **Camadas sem nome no Inspetor.** Os nomes foram escritos em **2D Render**, não em **2D Physics**.
- **Mexeu no inimigo e o jogador quebrou.** A edição foi feita com a aba do `jogador.tscn` na frente. Volte com Ctrl+Z nessa aba e salve.
- **"Cannot call method 'get_children' on a null value".** O nó não se chama `Nascedouros`.
- **Horda empilhada num ponto só.** A Mask do inimigo não tem a camada 3.
- **Inimigo empurrando você.** A Mask do inimigo ficou com a camada 2; ele deve passar por dentro.
- **Nunca perde vida.** A Mask da `Hurtbox` não tem a camada 3, ou o `CollisionShape2D` dela ficou fora da `Hurtbox` na árvore.
- **Morre de uma vez.** Faltou o `invencivel = true` antes do `await`.
- **"Identifier 'vida' not declared in the current scope".** Ao montar o `jogador.gd` aos pedaços, ficou de fora a linha `@export var vida: int = 3`, logo abaixo da `velocidade`.
- **A pergunta do toque em dois lugares.** O `if not invencivel...` vai só no `_process`. Colado também no `_physics_process`, não dá erro, mas na próxima mudança você corrige um e esquece o outro.
- **Erro em `instantiate()`.** O campo **Cena Inimigo** da `Vila` ficou vazio.
- **Erro "Nonexistent function 'perder'".** A função está no `vila.gd`, e o script não está ligado à `Vila`.
- **Inimigo nascendo preso.** O `Marker2D` caiu dentro de casa, árvore ou muro.
- **"Mixed use of tabs and spaces".** O código colado do PDF veio com espaços no lugar de Tab. Apague o recuo das linhas coladas e refaça com a tecla Tab.
- **Enter não recomeça.** O `fim.gd` não foi ligado ao nó `Fim` de uma das duas cenas.
- **Tempo zera e nada acontece.** O `timeout` do `Relogio` não foi conectado à `Vila`.
- **Erro de `null instance` ao abrir a vila.** Algum nó tem outro nome: `Jogador`, `HUD`, `Vida` ou `Tempo`. Renomeie o nó; o script fica como está.
- **A partida acaba e o jogo trava na vila.** A `derrota.tscn` ou a `vitoria.tscn` foi salva fora de `scenes/`. Mova pelo FileSystem, arrastando para a pasta certa.

O que você construiu hoje

A partida completa: alguém que ameaça, a pressão que cresce, um jeito de perder e um jeito de vencer. A partir daqui o jogo já se joga; as próximas aulas deixam o inimigo mais esperto (Aula 10), trocam o boneco vermelho pela arte do seu tema (Aula 11) e tiram o jogo do editor (Aula 12).

Próxima etapa — as fichas dos outros tipos

Estas fichas não fazem parte da entrega de hoje. Elas partem do jogo da tag `aula-09` e vão ser retomadas em aula; o ataque, que Abate e Chefe pedem e que Defesa usa, ainda vai ser demonstrado. Quem quiser adiantar pode começar, sem mexer na tag de hoje.

Cada ficha diz o que o `Relogio` faz no seu tipo, as peças novas e o código. Os trechos se somam ao que você já tem; quando uma função substitui outra, a ficha avisa.

Tipo	O `Relogio`	Peça nova
Sobrevivência	180 s e vence	nenhuma: é a entrega de hoje
Abate	desligado	ataque e contador
Chefe	60 s e o chefe surge	ataque e um inimigo grande
Coleta	desligado	item que nasce em pontos sorteados
Defesa	180 s e vence	objeto que os inimigos atacam
Fuga	60 s e a saída abre	saída escondida
Território	desligado	zona que conta tempo e muda de lugar
Resgate	desligado	aliados que seguem em fila

"Desligado" quer dizer: no `Relogio`, desmarque **Autostart**. A linha do `texto_tempo` passa a mostrar 0; troque-a pelo seu contador, como cada ficha mostra.

Abate — derrotar 50 inimigos

O ataque. **Project Settings** → **Input Map**, ação `atacar` com a tecla Espaço. Filho do `Jogador`: uma `Area2D` chamada `Ataque` (Layer nenhuma, Mask só na 3) com um `CircleShape2D` de raio 20. No `jogador.gd`, dentro do `_process`, abaixo do `levar_dano()`:

```
    if Input.is_action_just_pressed("atacar"):
        for inimigo in $Ataque.get_overlapping_bodies():
            inimigo.levar_golpe()
```

No `inimigo.gd`, abaixo da `velocidade`, e a função no fim do arquivo:

```
@export var vida: int = 1


func levar_golpe() -> void:
    vida -= 1
    if vida <= 0:
        get_tree().current_scene.inimigo_morreu(self)
        queue_free()
```

No `vila.gd`, no fim:

```
var abates := 0


func inimigo_morreu(_inimigo: Node2D) -> void:
    abates += 1
    if abates >= 50:
        vencer()
```

`Relogio` desligado; no placar, `texto_tempo.text = "Abates:  %d" % abates`.

Chefe — resistir até o chefe surgir e derrotá-lo

O mesmo ataque do Abate: a ação `atacar`, a área `Ataque`, o trecho no `jogador.gd` e o `levar_golpe` no `inimigo.gd`. O chefe é o mesmo inimigo, maior e com mais vida. No `vila.gd`, substitua a função `_on_relogio_timeout` por esta, e acrescente a `inimigo_morreu`:

```
var chefe: Node2D = null


func _on_relogio_timeout() -> void:
    chefe = cena_inimigo.instantiate()
    chefe.vida = 10
    chefe.scale = Vector2(2, 2)
    chefe.global_position = $Nascedouros/Cima.global_position
    add_child(chefe)


func inimigo_morreu(inimigo: Node2D) -> void:
    if inimigo == chefe:
        vencer()
```

`Relogio` em 60 s: é a contagem até o chefe aparecer.

Coleta — juntar 20 itens

O item é uma cena nova: `Area2D` chamada `Item` (Layer nenhuma, Mask só na 2), com um `Sprite2D` (o `tiny-town.png` com **Region** ligada num cogumelo, `Rect2(80, 32, 16, 16)`, ou qualquer arte) e um `CircleShape2D` de raio 6. Salve em `entities/item.tscn` e conecte o `body_entered` da `Item` a

ela mesma. Script `scripts/item.gd`:

```
extends Area2D


func _on_body_entered(_body: Node2D) -> void:
    get_tree().current_scene.coletou()
    queue_free()
```

Na vila: um `Node2D` chamado `Pontos` com uns oito `Marker2D` espalhados em chão livre, e um `Timer` chamado `Itens` (Wait Time 3, Autostart), com o `timeout` conectado à `Vila`. No `vila.gd`, abaixo da `cena_inimigo`, `@export var cena_item: PackedScene` (arraste o `item.tscn` no Inspetor) e, no fim:

```
var coletados := 0


func _on_itens_timeout() -> void:
    var item := cena_item.instantiate()
    item.global_position = $Pontos.get_children().pick_random().global_position
    add_child(item)


func coletou() -> void:
    coletados += 1
    if coletados >= 20:
        vencer()
```

`Relogio` desligado; no placar, `"Itens: %d / 20" % coletados`.

Defesa — proteger um objeto por 3 minutos

O objeto é uma cena nova: `Area2D` chamada `Objeto`, no grupo `objeto` (Layer nenhuma, Mask só na 3), com um `Sprite2D` (o poço, `Rect2(128, 128, 16, 16)`) e um `CircleShape2D` de raio 10. Conecte o `body_entered` a ela mesma. Script `scripts/objeto.gd`:

```
extends Area2D

var toques := 0


func _on_body_entered(inimigo: Node2D) -> void:
    inimigo.queue_free()
    toques += 1
    if toques >= 5:
        get_tree().current_scene.perder()
```

Ponha um `Objeto` na vila, em chão livre. Agora o inimigo persegue o objeto, e não você. No `inimigo.gd`, abaixo da `velocidade`, `@export var grupo_alvo: String = "player"`, e troque `get_first_node_in_group("player")` por `get_first_node_in_group(grupo_alvo)`. No `vila.gd`, na `_on_horda_timeout`, antes do `add_child`:

```
    inimigo.grupo_alvo = "objeto"
```

`Relogio` em 180 s. Para você poder fazer alguma coisa, dê ao jogador o ataque do Abate, com uma `inimigo_morreu` vazia no `vila.gd` (só com `pass`).

Fuga — sobreviver até a saída abrir, e alcançá-la

Na vila: uma `Area2D` chamada `Saida` (Layer nenhuma, Mask só na 2), longe do jogador, com um `Sprite2D` e um `CircleShape2D` de raio 10. No Inspetor dela, desmarque **Visible** e **Monitoring**: a saída começa invisível e sem perceber ninguém. Conecte o `body_entered` da `Saida` à `Vila`. No `vila.gd`, substitua a `_on_relogio_timeout` por estas duas:

```
func _on_relogio_timeout() -> void:
    $Saida.visible = true
    $Saida.monitoring = true


func _on_saida_body_entered(_body: Node2D) -> void:
    vencer()
```

`Relogio` em 60 s: é a contagem até a saída abrir.

Território — somar 60 segundos dentro da zona

Na vila: uma `Area2D` chamada `Zona` (Layer nenhuma, Mask só na 2), com um `ColorRect` amarelo transparente de 48 × 48 centrado nela e um `RectangleShape2D` do mesmo tamanho. Um `Node2D` chamado `Pontos` com `Marker2D` em chão livre, e um `Timer` chamado `MudaZona` (Wait Time 10, Autostart), com o `timeout` conectado à `Vila`. No `vila.gd`, no fim:

```
var tempo_na_zona := 0.0


func _physics_process(delta: float) -> void:
    if $Zona.has_overlapping_bodies():
        tempo_na_zona += delta
        if tempo_na_zona >= 60:
            vencer()


func _on_muda_zona_timeout() -> void:
    $Zona.global_position = $Pontos.get_children().pick_random().global_position
```

`Relogio` desligado; no placar, `"Zona:  %d / 60" % tempo_na_zona`.

Resgate — encontrar 5 aliados, que seguem em fila

O aliado é uma cena nova: `Area2D` chamada `Aliado` (Layer nenhuma, Mask na 2 e na 3), com um `Sprite2D` (o personagem com **Modulate** azul) e um `CircleShape2D` de raio 6. Conecte o `body_entered` a ela mesma. Script `scripts/aliado.gd`:

```
extends Area2D

var lider: Node2D = null


func _physics_process(delta: float) -> void:
    if lider and global_position.distance_to(lider.global_position) > 14:
        global_position = global_position.move_toward(lider.global_position, 70 * delta)


func _on_body_entered(body: Node2D) -> void:
    if body.is_in_group("player") and lider == null:
        get_tree().current_scene.resgatou(self)
    elif body.is_in_group("inimigo") and lider != null:
        get_tree().current_scene.perder()
```

Espalhe cinco `Aliado` pela vila, em chão livre. No `vila.gd`, no fim:

```
var fila: Array[Node2D] = []


func resgatou(aliado: Node2D) -> void:
    if fila.is_empty():
        aliado.lider = jogador
    else:
        aliado.lider = fila.back()
    fila.append(aliado)
    if fila.size() >= 5:
        vencer()
```

O primeiro aliado segue você; cada um dos seguintes segue o anterior. É a cobrinha: `lider` é quem vai na frente. `Relogio` desligado; no placar, `"Aliados:  %d / 5" % fila.size()`.